CALL LOG MERGER

USING SINGLY LINKED LIST

PROJECT REPORT

Subject	Data Structures and Algorithms
Programming Language	C++
Data Structure	Singly Linked List
Project Title	Call Log Merger

Department of Electronics and Telecommunication Engineering
Amrutvahini College of Engineering, Sangamner

Submitted by: **Ritesh Ramrao Bhalerao**
Second Year, Electronics and Telecommunication Engineering

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope and Applications
6. Software and Hardware Requirements
7. Theory: Singly Linked List
8. Proposed Method
9. Algorithm
10. Flowchart
11. C++ Source Code
12. Sample Output
13. Explanation of Output
14. Complexity Analysis
15. Advantages
16. Limitations
17. Future Enhancements
18. Testing
19. Conclusion
20. References

1. Abstract

The Call Log Merger project demonstrates the use of a singly linked list to combine two sets of phone-call records into one list. Each node stores a call record and a pointer to the next node. The program creates two linked lists, joins them in the required order, and displays the merged call log. The project helps in understanding dynamic memory allocation, pointers, node creation, traversal, and linked-list concatenation.

2. Introduction

A data structure is a way of organizing and storing data so that it can be used efficiently. A linked list is a linear data structure made of nodes that are connected through pointers. Unlike an array, the nodes of a linked list do not need to occupy consecutive memory locations, so the list can grow or shrink at run time without moving existing data.

In a singly linked list, every node contains data and a pointer to the next node. In this project, two call logs are represented as separate singly linked lists and then combined into one list.

3. Problem Statement

A phone application stores call records in two separate linked lists. Develop a C++ program that combines the two lists into a single linked list while preserving the order of records within each list. Finally, display the merged call log.

4. Objectives

- Understand the structure and working of a singly linked list.
- Create and connect nodes dynamically using pointers.
- Store call-log records in two separate lists.
- Merge the lists while preserving their original order.
- Traverse and display the resulting list.
- Release dynamically allocated memory correctly.
- Analyze the time and space complexity of the solution.

5. Scope and Applications

This is an educational model of merging sequential records. The same concept can be used in contact management, activity logs, transaction records, event histories, and for combining data collected by separate modules. A real phone application would require additional fields such as caller number, date, time, call duration, and call type, along with input validation and persistent storage.

6. Software and Hardware Requirements

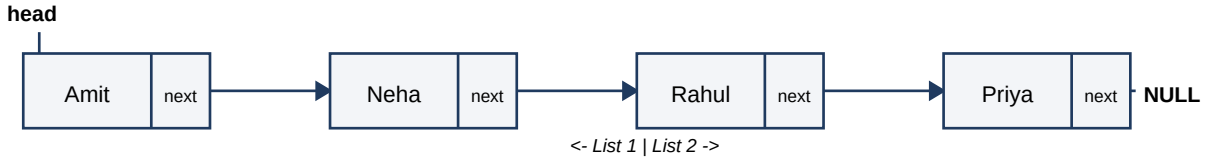
Item	Requirement
Hardware	Computer or laptop
Operating system	Windows, Linux, or macOS
Software	C++ compiler such as GCC or MinGW, or an IDE such as Visual Studio Code, Code::Blocks, or Dev-C++

Item	Requirement
Language	C++ (C++11 or later)
Data structure	Singly linked list

7. Theory: Singly Linked List

A **node** is the basic unit of a linked list. In this project each node contains a call-record string and a pointer named `next`. The **head** pointer identifies the first node, and the last node has `next = nullptr`, which marks the end of the list. Traversal starts at the head and follows the `next` pointers until `nullptr` is reached.

Node representation of the merged list (first two nodes belong to List 1, the last two to List 2):



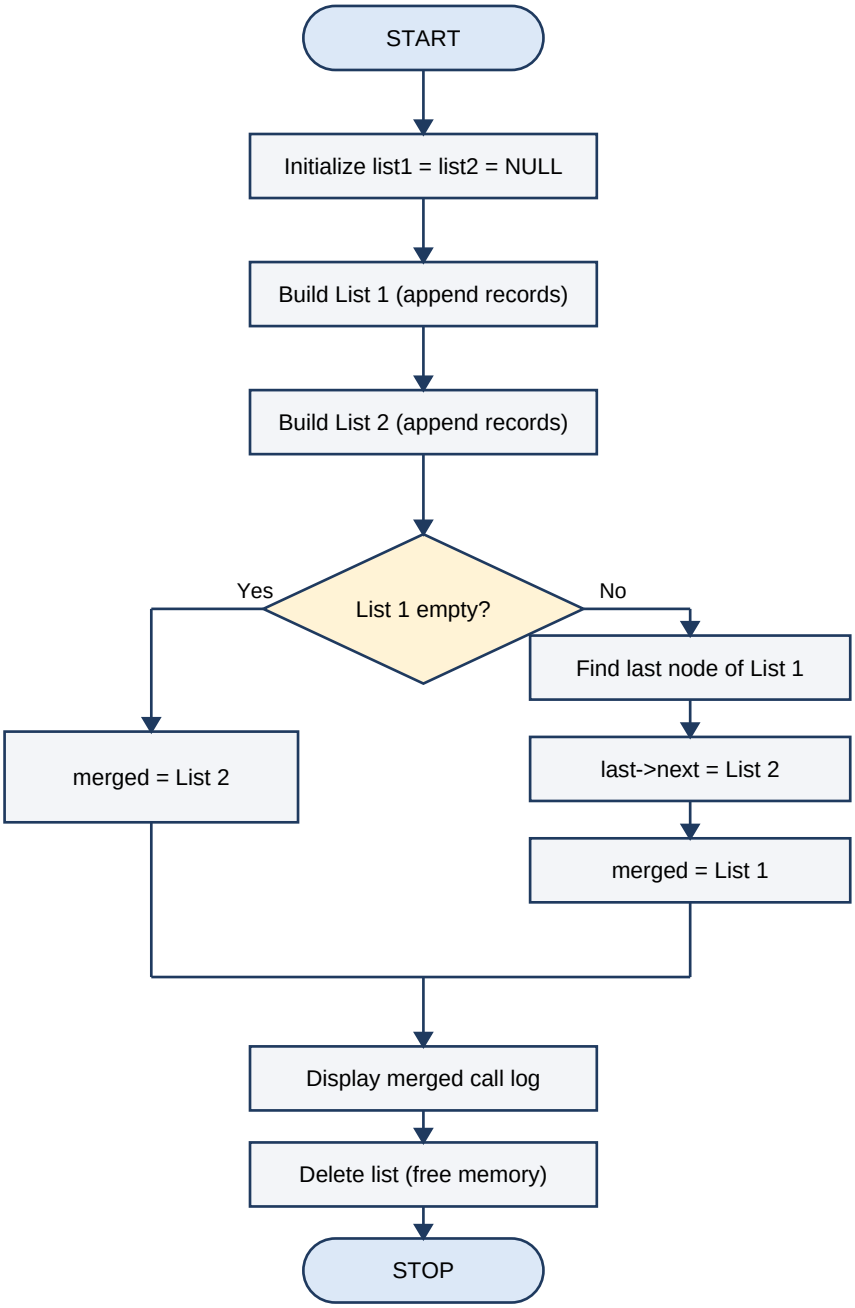
8. Proposed Method

The program builds List 1 and List 2. If List 1 is empty, the head of List 2 becomes the head of the merged list. Otherwise, the program traverses List 1 to its final node and sets that node's `next` pointer to the head of List 2. The merged list is then traversed to display all records. This method **concatenates** the lists; it does not sort them by date or time.

9. Algorithm

1 Start.
2 Initialize the head pointers of List 1 and List 2 to `nullptr`.
3 Create nodes and append the required records to List 1.
4 Create nodes and append the required records to List 2.
5 If List 1 is empty, set the merged head to List 2.
6 Otherwise, move a pointer through List 1 until its last node is found.
7 Connect the last node of List 1 to the head of List 2.
8 Traverse the merged list and display each record.
9 Release all allocated memory and stop.

10. Flowchart



11. C++ Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string callRecord;
    Node* next;
    Node(const string& record) : callRecord(record), next(nullptr) {}
};

void append(Node*& head, const string& record) {
    Node* newNode = new Node(record);
    if (head == nullptr) {
        head = newNode;
        return;
    }
    Node* current = head;
    while (current->next != nullptr) {
        current = current->next;
    }
    current->next = newNode;
}

Node* mergeLists(Node* first, Node* second) {
    if (first == nullptr) return second;
    Node* current = first;
    while (current->next != nullptr) {
        current = current->next;
    }
    current->next = second;
    return first;
}

void display(Node* head) {
    if (head == nullptr) {
        cout << "Call log is empty.\n";
        return;
    }
    Node* current = head;
    while (current != nullptr) {
        cout << current->callRecord << '\n';
        current = current->next;
    }
}

void deleteList(Node*& head) {
    while (head != nullptr) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

int main() {
    Node* list1 = nullptr;
    Node* list2 = nullptr;

    append(list1, "Amit - 10:15 AM - Incoming");
    append(list1, "Neha - 10:30 AM - Outgoing");
    append(list2, "Rahul - 11:00 AM - Missed");
    append(list2, "Priya - 11:20 AM - Incoming");
```

```
    cout << "List 1:\n";
    display(list1);
    cout << "\nList 2:\n";
    display(list2);

    Node* merged = mergeLists(list1, list2);
    // Ownership of both lists is now represented by merged.
    list1 = nullptr;
    list2 = nullptr;

    cout << "\nMerged Call Log:\n";
    display(merged);

    deleteList(merged);
    return 0;
}
```

12. Sample Output

```
List 1:
Amit - 10:15 AM - Incoming
Neha - 10:30 AM - Outgoing

List 2:
Rahul - 11:00 AM - Missed
Priya - 11:20 AM - Incoming

Merged Call Log:
Amit - 10:15 AM - Incoming
Neha - 10:30 AM - Outgoing
Rahul - 11:00 AM - Missed
Priya - 11:20 AM - Incoming
```

13. Explanation of Output

The first list contains two records and the second list also contains two records. The final node of the first list is connected to the first node of the second list. Therefore, the merged list displays the two records from List 1 followed by the two records from List 2. The original order inside each list is preserved.

14. Complexity Analysis

Operation	Time	Explanation
Append one record	O(n)	The function traverses to the last node.
Merge two lists	O(n)	The first list is traversed to its last node.
Display merged list	O(n + m)	Every node in both lists is visited.
Extra space for merge	O(1)	Only a constant number of pointers is used.

Here *n* is the number of nodes in the first list and *m* is the number of nodes in the second list. If a tail pointer is maintained for List 1, concatenation can be performed in O(1) time; this implementation searches for the tail, so merging takes O(n).

15. Advantages

- Linked lists grow dynamically as records are added.
- Nodes do not need contiguous memory locations.
- Concatenation is done by changing a single pointer; no record is copied.
- The project provides practical experience with pointers and dynamic memory.

16. Limitations

- The program uses sample records and does not accept interactive user input.
- It concatenates lists but does not sort records chronologically.
- It does not save call records permanently to a file or database.
- Each node requires memory for a pointer in addition to the record.

17. Future Enhancements

- Accept call records from the user.
- Store caller number, date, time, duration, and call type as separate fields.
- Sort merged records by date and time when required.
- Add search, deletion, and duplicate-record detection.
- Save and load call logs using file handling or a database.
- Use a tail pointer to make list concatenation faster.

18. Testing

The program was compiled with `g++ -Wall` and tested with the following cases. Memory behaviour was also checked with AddressSanitizer, which reported no errors or leaks.

Test case	Input	Result
Both lists non-empty	A1, A2 and B1, B2	A1, A2, B1, B2
First list empty	(empty) and B1, B2	B1, B2
Second list empty	A1, A2 and (empty)	A1, A2
Both lists empty	(empty) and (empty)	Call log is empty.

In every case no records were lost, the order inside each list was preserved, and no null pointer was dereferenced.

19. Conclusion

The Call Log Merger project demonstrates how a singly linked list can combine two ordered groups of records. By connecting the final node of the first list to the beginning of the second list, the program creates one merged list without copying any record into new nodes. The project strengthens understanding of linked-list traversal, pointers, dynamic memory allocation, and complexity analysis.

20. References

- Data Structures and Algorithms course notes.

- C++ language reference and documentation (cppreference.com).
- Classroom material on singly linked lists and pointer operations.